# Архитектура ЭВМ и язык ассемблера

Семинар #34:

1. Аппаратный стек, инструкции `push` и `pop`, `call` и `ret`.
2. Соглашение о вызове функции: `cdecl`.
3. Вспомогательные макросы для вызова функций.
4. Примеры вызова функций.

13.03.2026

# Аппаратный стек, инструкции PUSH и POP, CALL и RET

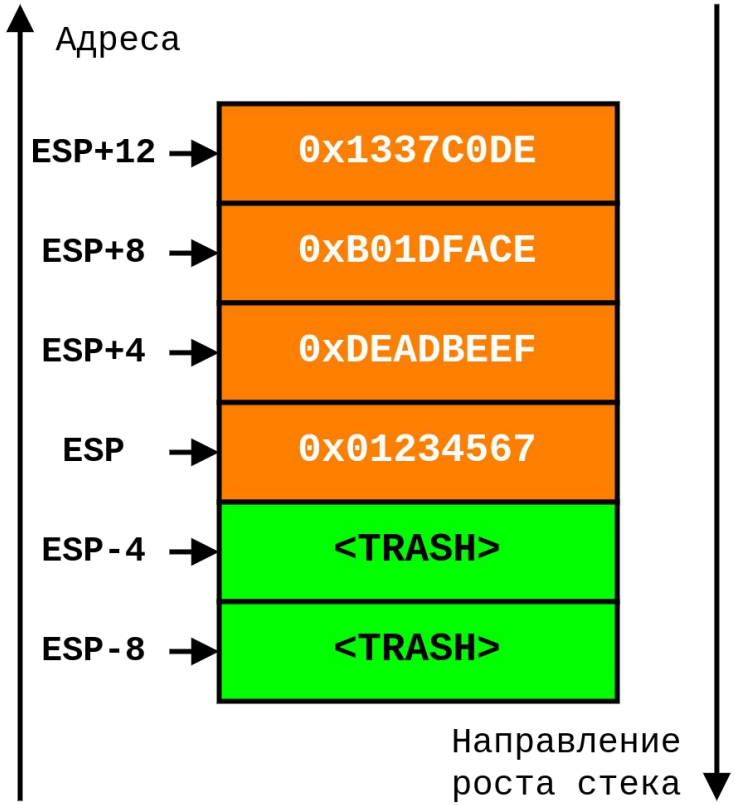
# Аппаратный стек

**Инициализация стека:**

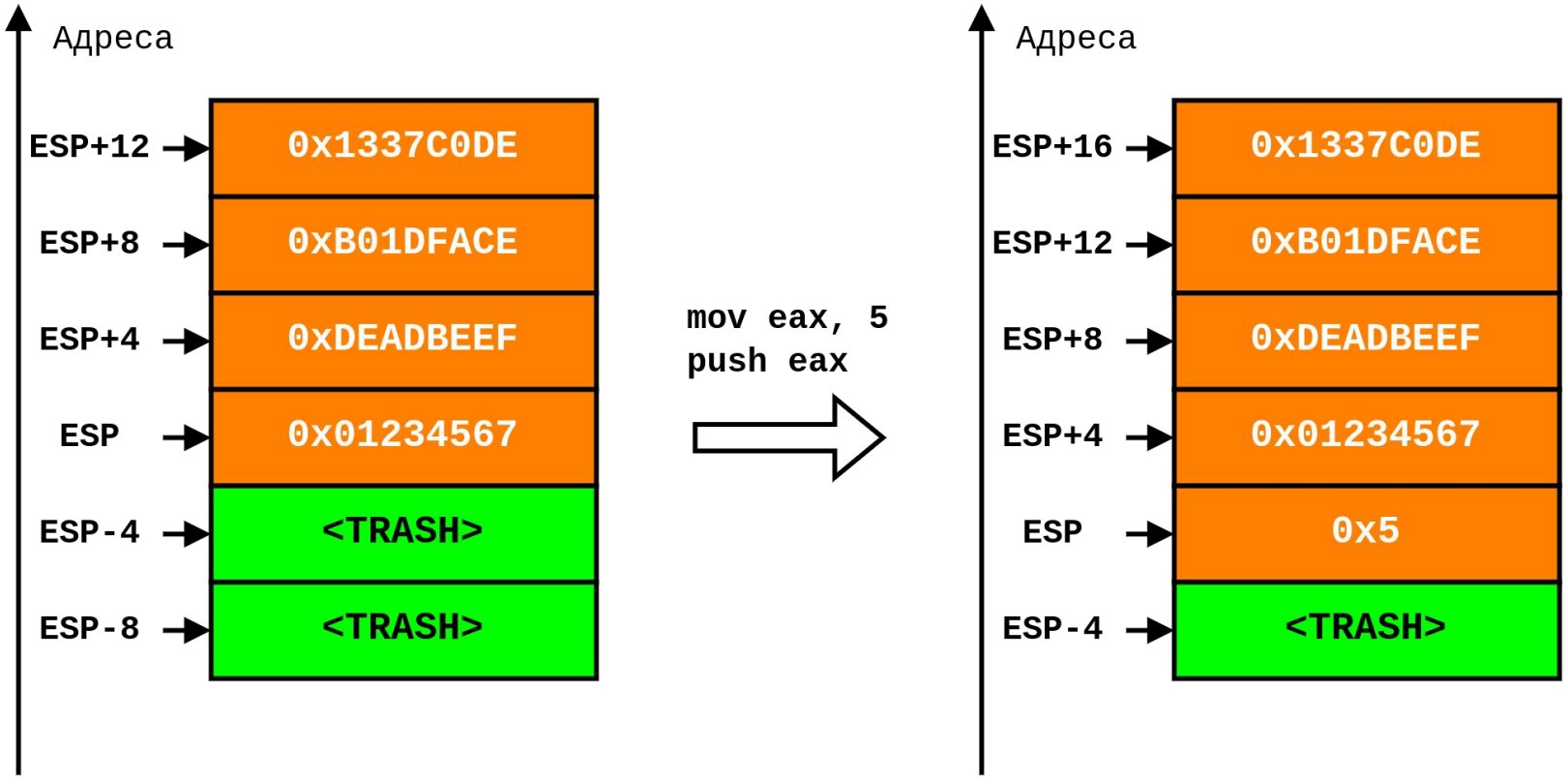
- Выполняется операционной системой при запуске программы.
- В функции _start `ESP ≈ 0xFFFFC000`

**Применения стека:**

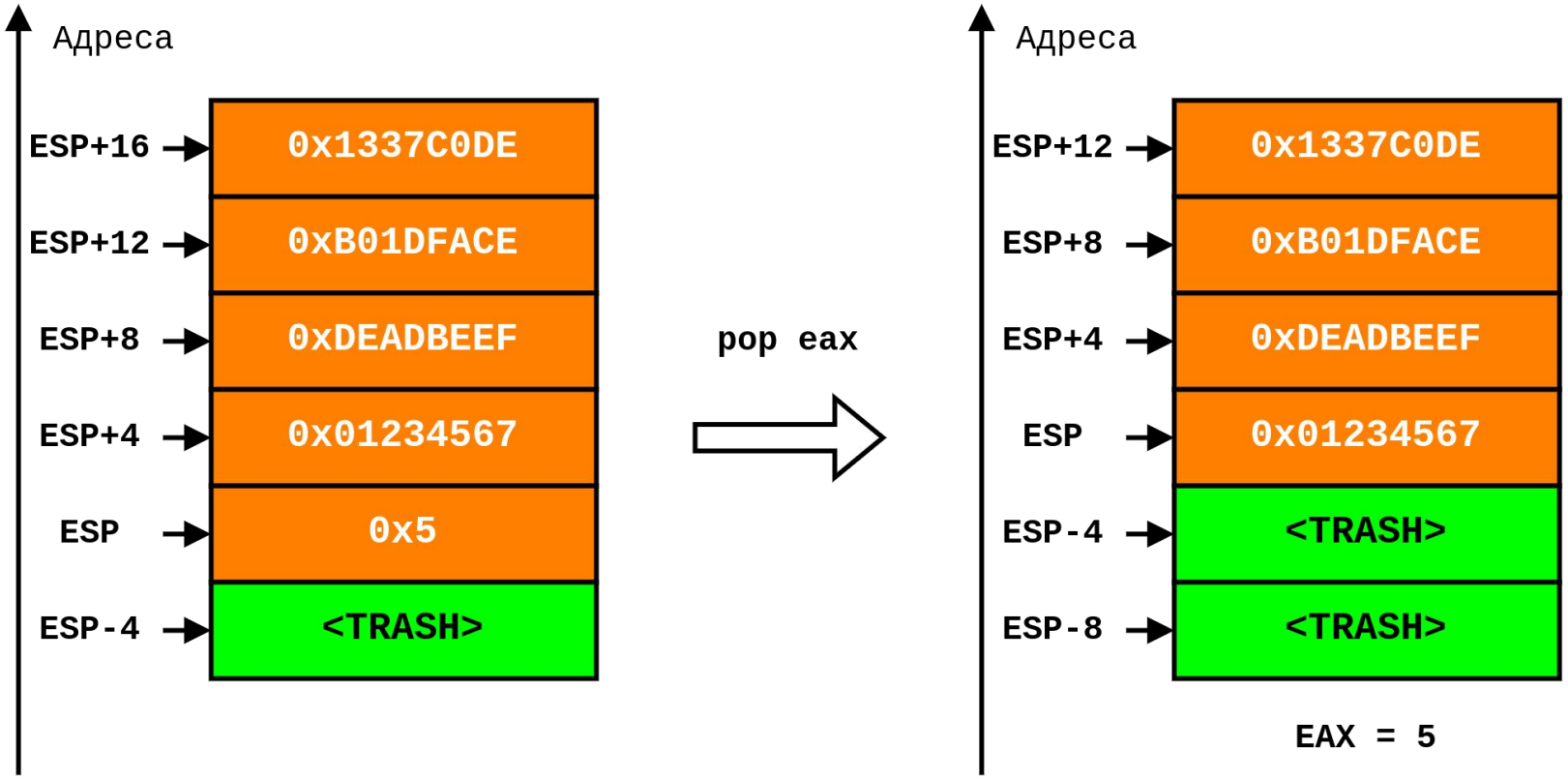
- Хранение автоматических локальных переменных.
- Передача аргументов в функцию.
- Хранение адреса возврата.



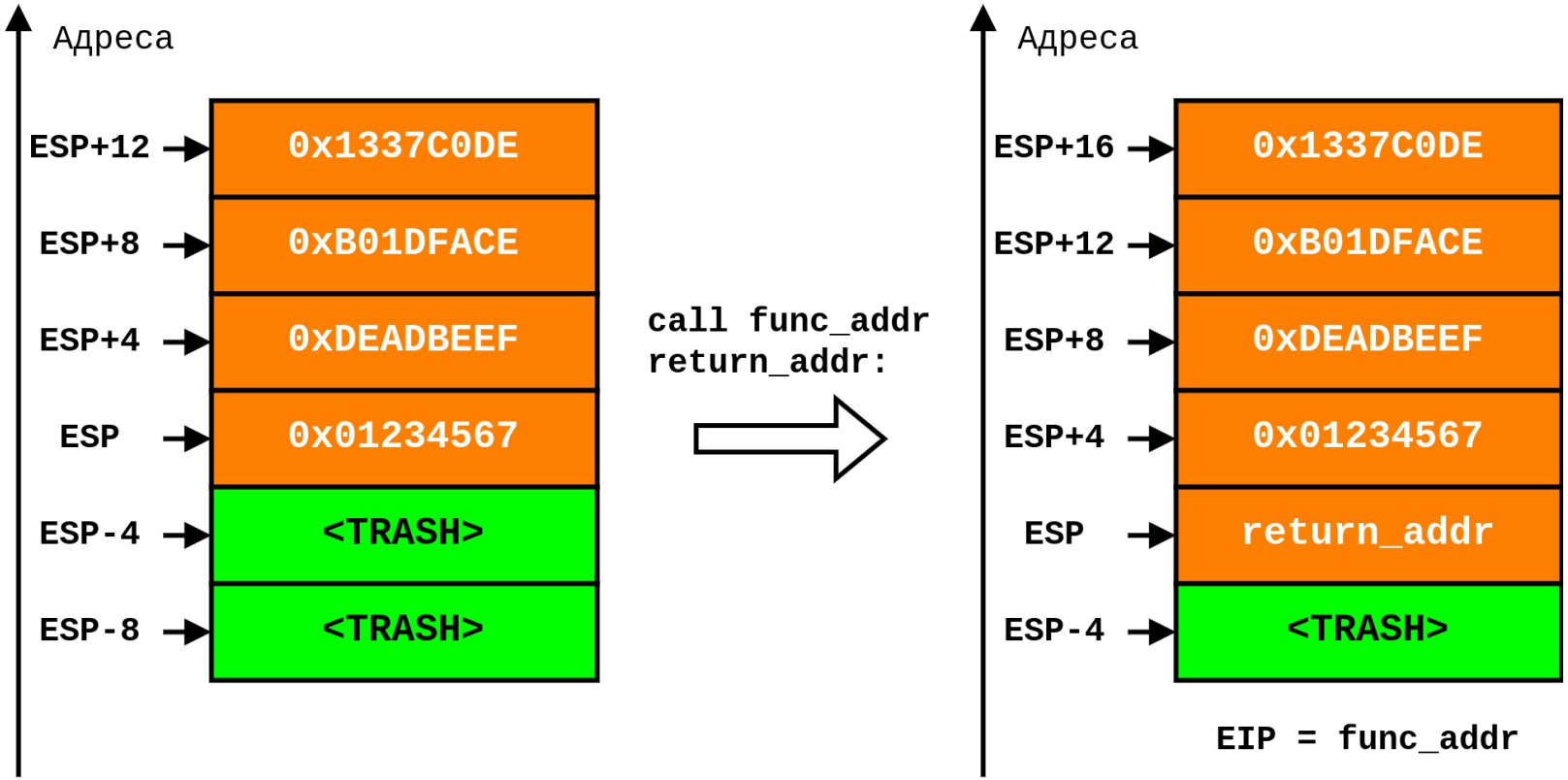
# Инструкция push



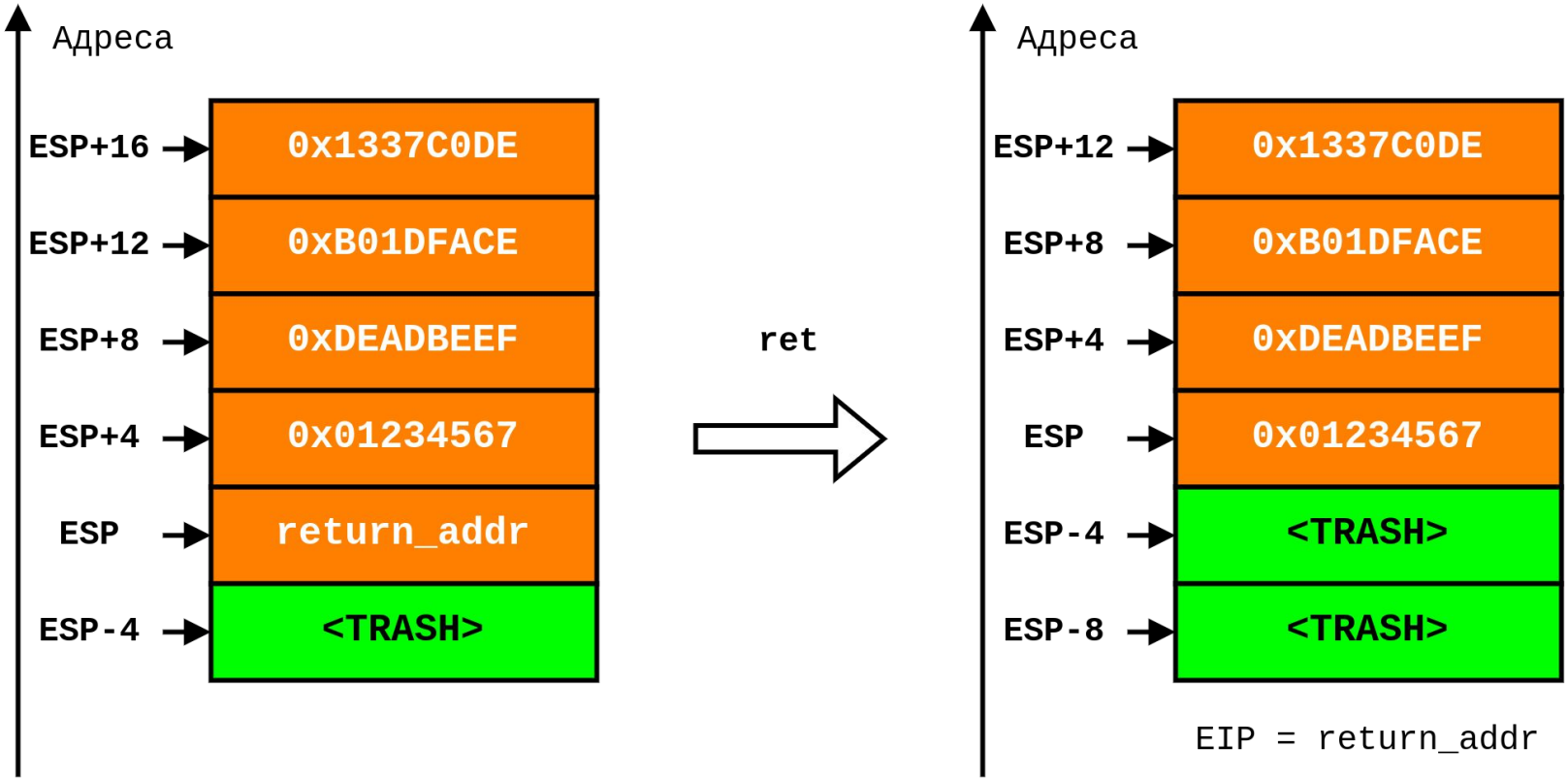
# Инструкция pop



# Инструкция call



# Инструкция ret



# Соглашение о вызове функции: cdecl

# Сохранение регистров при вызове функции

Table 2.3: Register Usage

| Register | Usage | Preserved across function calls |
|---|---|---|
| %eax | scratch register; also used to return integer and pointer values from functions; also stores the address of a returned struct or union | No |
| %ebx | callee-saved register; also used to hold the GOT pointer when making function calls via the PLT | Yes |
| %ecx | scratch register | No |
| %edx | scratch register; also used to return the upper 32bits of some 64bit return types | No |
| %esp | stack pointer | Yes |
| %ebp | callee-saved register; optionally used as frame pointer | Yes |
| %esi | callee-saved register | yes |
| %edi | callee-saved register | yes |

System V i386 ABI -> Function Calling Sequence -> 2.3.1. Registers

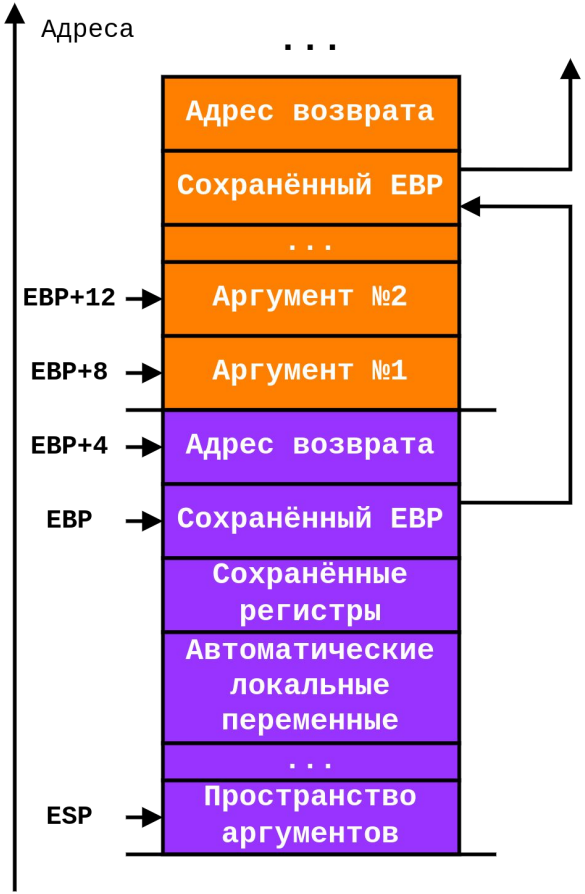
# Формат стекового фрейма

**При вызове функции:**

1. Выравниванием стек на 16 байт.
2. Формируем аргументы функции.
3. Сохраняем регистры `EAX, ECX, EDX` (если они использовались).
4. Делаем вызов функции с помощью инструкции `call`.

**Пролог вызванной функции:**

1. Сохраняет и обновляет EBP.
2. Сохраняет регистры `ESI, EDI, EBX` (если они будут использоваться).
3. Резервирует место для переменных.



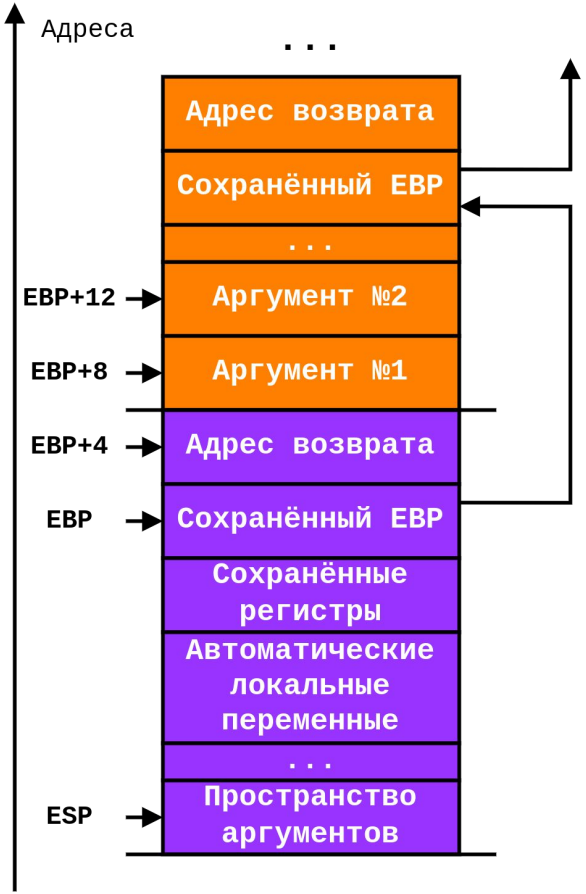
# Формат стекового фрейма

**Эпилог вызванной функции:**

1. Восстанавливает регистры `ESI, EDI, EBX.`
2. Освобождает память стека из под локальных переменных.
3. Восстанавливает EBP.
4. Исполняет инструкцию `ret`.

**После вызова функции:**

1. Освобождаем память стека под аргументы функции.
2. Восстанавливаем регистры `EAX, ECX, EDX` (если их сохраняли).

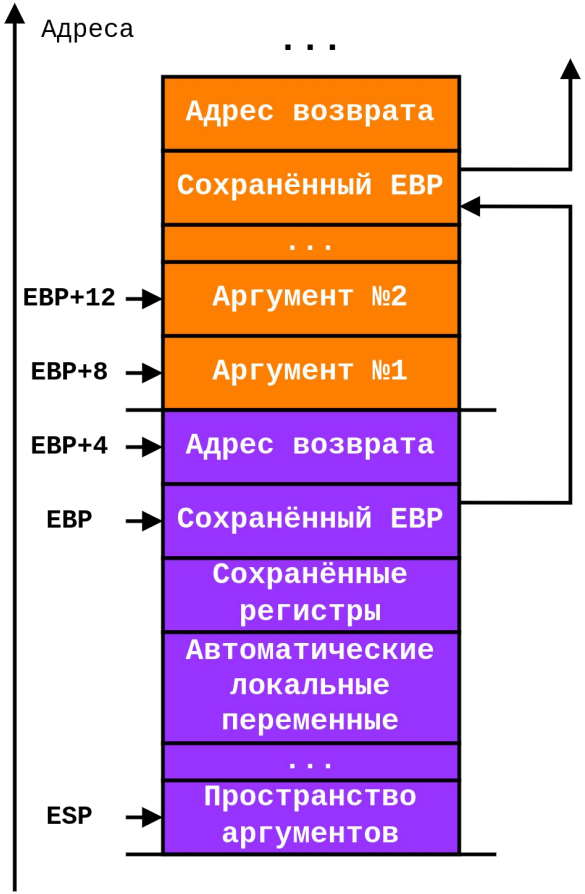


# Вспомогательные макросы для вызова функций

# Организация вызова функции

```
%macro ALIGN_STACK 1.nolist
    sub     esp, %1
    and     esp, 0xfffffff0
    add     esp, %1
%endmacro

%macro UNALIGN_STACK 1.nolist
    add     esp, %1
%endmacro
```

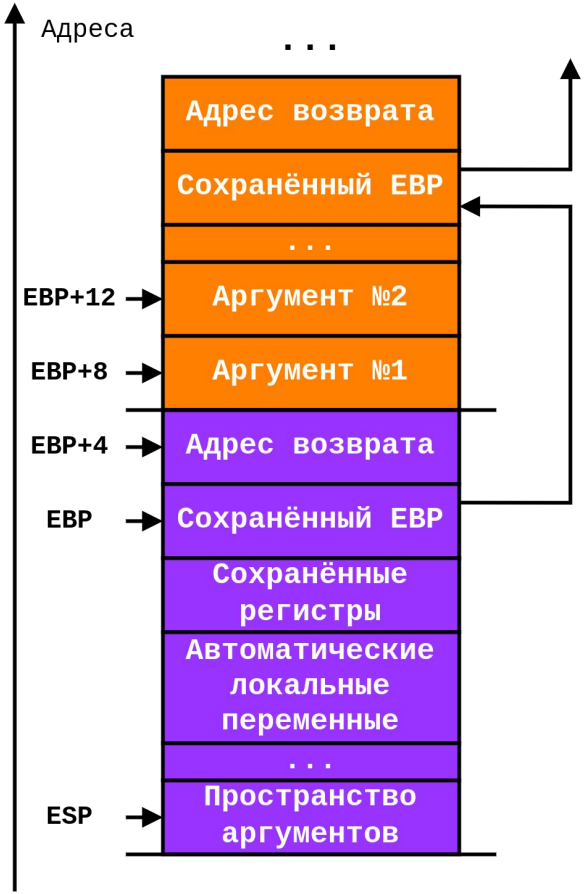
```
; Инициализируем матрицу A.
ALIGN_STACK 12
push MATRIX_SIZE ; size_y
push MATRIX_SIZE ; size_x
push mat_A       ; matrix_base
call matrix_init
UNALIGN_STACK 12
```



# Пролог и эпилог функции

```
%macro FUNCTION_PROLOGUE 1.nolist
    push  ebp
    mov   ebp, esp
    sub   esp, %1
%endmacro

%macro FUNCTION_EPILOGUE 0.nolist
    mov   esp, ebp
    pop   ebp
%endmacro
```

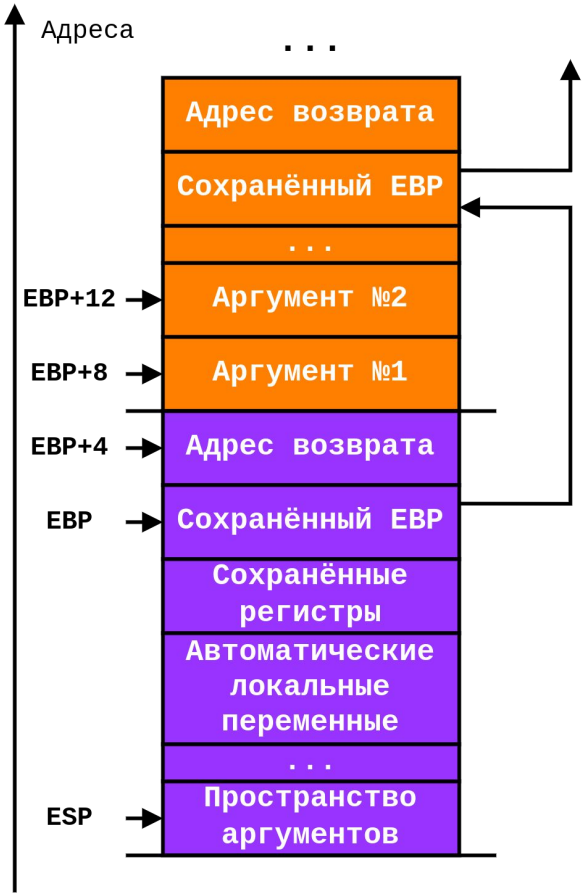
```
%macro FUNCTION_PROLOGUE 1.nolist
    enter   %1, 0
%endmacro

%macro FUNCTION_EPILOGUE 0.nolist
    leave
%endmacro
```



# Пролог функции

```
%macro FUNCTION_PROLOGUE 1.nolist
    push  ebp
    mov   ebp, esp
    sub   esp, %1
%endmacro
```

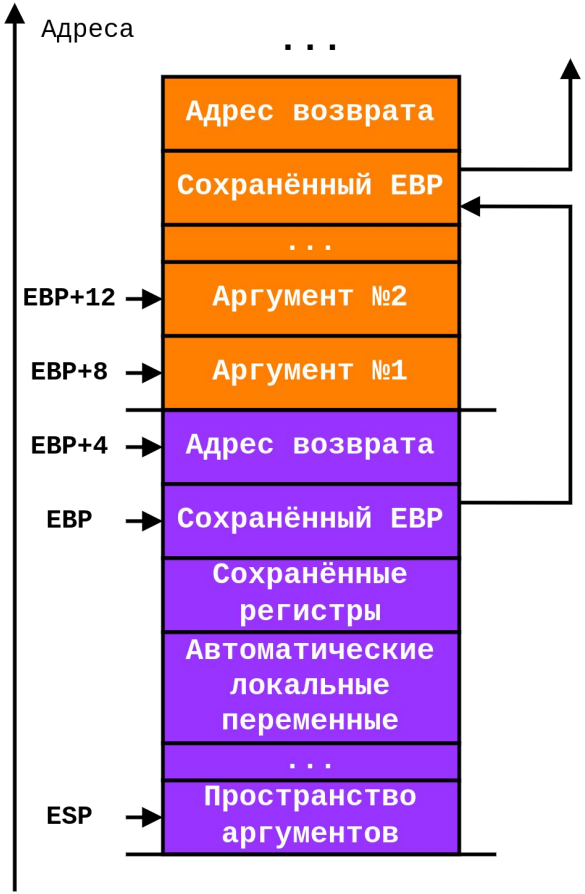
```
global matrix_init
%define size_y          dword [ebp + 16]
%define size_x          dword [ebp + 12]
%define matrix_base     dword [ebp +  8]
%define tmp_ebx         dword [ebp -  4]
%define tmp_edi         dword [ebp -  8]
%define tmp_esi         dword [ebp - 12]
matrix_init:
    ; Инициализируем стековый фремй
    FUNCTION_PROLOGUE 12
    ; Сохраняем callee-preserved регистры.
    mov     tmp_ebx, ebx
    mov     tmp_edi, edi
    mov     tmp_esi, esi
```



# Эпилог функции

```
%macro FUNCTION_EPILOGUE 0.nolist
    mov    esp, ebp
    pop    ebp
%endmacro
```

```
    ; Восстанавливаем callee-preserved регистры.
    mov     ebx, tmp_ebx
    mov     edi, tmp_edi
    mov     esi, tmp_esi

    ; Восстанавливаем стековый фрейм.
    FUNCTION_EPILOGUE
    ; Возвращаемся из функции.
    ret

; Удаляем макросы во избежание ошибок.
%undef size_y
%undef size_x
%undef matrix_base
%undef tmp_ebx
%undef tmp_edi
%undef tmp_esi
```



# Примеры возова функций

# 32-битные числа в cdecl

```c
__attribute__((cdecl))
uint32_t addOne32(uint32_t arg)
{
    return arg + 1U;
}
```

```
push    ebp
mov     ebp, esp
mov     eax, dword [ebp + 0x8]
add     eax, 0x1
pop     ebp
ret
```

**Передача аргумента:**
размещение на стеке.

**Возвращаемое значение:**
регистр **EAX**.

```
push    0x0
call    addOne32
add     esp, 0x4
mov     dword [ebp - 0x10], eax
```

# 64-битные числа в cdecl

```c
__attribute__((cdecl))
uint64_t addOne64(uint64_t arg)
{
    return arg + 1U;
}
```

```
push    ebp
mov     ebp, esp
mov     eax, dword [ebp + 0x8]
mov     edx, dword [ebp + 0xC]
add     eax, 0x1
add     edx, 0x0
pop     ebp
ret
```

**Передача аргумента:**
размещение на стеке.

**Возвращаемое значение:**
регистровая паре **EDX:EAX**.

```
push    0x0
push    0x0
call    addOne64
add     esp, 0x8
mov     dword [ebp - 0x18], eax
mov     dword [ebp - 0x14], edx
```

# Структуры в cdecl

```c
typedef struct
{
    uint8_t array[8];
} Struct;

__attribute__((cdecl))
Struct addOneStruct(Struct arg)
{
    arg.array[0] += 1;
    return arg;
}
```

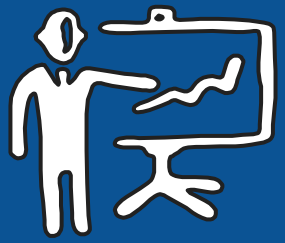
```
push    ebp {__saved_ebp}
mov     ebp, esp {__saved_ebp}
movzx   eax, byte [ebp+0xc {arg2}]
add     eax, 0x1
mov     byte [ebp+0xc {arg2}], al
mov     ecx, dword [ebp+0x8 {arg1}]
mov     eax, dword [ebp+0xc {arg2}]
mov     edx, dword [ebp+0x10 {arg3}]
mov     dword [ecx], eax
mov     dword [ecx+0x4], edx
mov     eax, dword [ebp+0x8 {arg1}]
pop     ebp {__saved_ebp}
retn    0x4 {__return_addr}
```

**Передача аргумента:**
размещение на стеке.

**Возвращаемое значение:**
запись по указателю, передаваемому первым аргументов.

```
lea     eax, [ebp-0x30 {var_38}]
sub     esp, 0x4
push    dword [ebp-0x1c {var_24}]
push    dword [ebp-0x20 {var_28}]
push    eax {var_38} {var_48}
call    addOneStruct
add     esp, 0xc
```

# Вопросы?



Красивые иконки взяты с сайта handdrawngoods.com